

Retrieval-Augmented Generation (RAG)

RAG combines a large language model with an external knowledge source. Instead of relying only on model parameters, user queries are used to retrieve relevant documents which are supplied as context to the LLM. This improves factual accuracy, enables answers over private data, and reduces hallucinations.

Typical RAG Pipeline

1. Load documents (PDFs, HTML, databases). 2. Split into chunks. 3. Generate embeddings. 4. Store embeddings in a vector index such as FAISS. 5. Embed the user query. 6. Retrieve the top-k relevant chunks. 7. Send the retrieved context plus the user query to the LLM. 8. Return the grounded answer with citations.

Sparse vs Dense Retrieval

Sparse retrieval (BM25) matches exact keywords using term frequency and inverse document frequency. Dense retrieval represents documents and queries as embeddings, allowing semantic matching even when the wording differs.

Hybrid Search

Hybrid search combines BM25 and dense vector search. BM25 excels at exact names, IDs, and keywords, while dense retrieval captures semantic similarity. Scores can be merged using weighted sums or reciprocal rank fusion (RRF). Hybrid search usually improves recall and robustness.

FAISS

FAISS is a high-performance library for vector similarity search. Common indexes include IndexFlatIP for cosine similarity (with normalized vectors), HNSW for approximate nearest-neighbor search, and IVF for large datasets.

Best Practices

Use chunk sizes around 300–1000 tokens depending on the content, preserve metadata such as source and page number, normalize embeddings for cosine similarity, rerank retrieved results when necessary, and include inline citations in generated answers.